

AutoMeKin — Complete Tutorial for Beginners

A from-zero, step-by-step walkthrough

AutoMeKin Documentation

September 17, 2026

Contents

Introduction	3
0.1 Contents	3
1 Installation from zero	4
1.1 What is AutoMeKin?	4
1.2 Prerequisites	4
1.3 Installation roadmap	4
1.4 Step 1 — System dependencies	5
1.5 Step 2 — Python with Miniconda	6
1.5.1 2.1 Install Miniconda (if not already present)	6
1.5.2 2.2 Create the environment for AutoMeKin	6
1.5.3 2.3 Install the required libraries	6
1.6 Step 3 — Download the source code	6
1.7 Step 4 — Build and install AutoMeKin	7
1.8 Step 5 — Set up the environment permanently	7
1.9 Step 6 — Verify that everything works	8
1.10 Step 7 — (Optional) High-level backends: ORCA, MACE and UMA	8
1.10.1 7.1 What are LL and HL, and when is this step needed?	8
1.10.2 7.2 ORCA	9
1.10.3 7.3 Machine-learning interatomic potentials (MLIP): MACE and UMA	12
1.11 Common troubleshooting	17
1.12 What's next	17
2 Low-level (LL) calculations	18
2.1 LL vs HL: what this page covers	18
2.2 Setting up the working folder	18
2.2.1 Starting structure	18
2.3 The <code>amk.dat</code> input file	19
2.4 Launching the calculation	20
2.5 What happens while it runs: the iteration loop	20
2.6 Where results actually live — and why <code>batch*</code> disappears	20
2.6.1 <code>tsdirLL_<molecule>/</code> — the working database (accumulates iteration after iteration)	21
2.6.2 <code>FINAL_LL_<molecule>/</code> — the curated final summary	21
2.7 Reading the final results	21
2.8 Full verified example: FA (formic acid)	21
2.9 Common troubleshooting	22
2.10 What's next	22
3 High-level (HL) calculations with ORCA	23
3.1 What the HL step does	23
3.2 Requirements before starting	23

3.3	Launching the calculation	24
3.4	What happens while it runs: <code>hlcalcs.sh</code> 's phases	24
3.5	Where results live	24
3.6	Reading the final results	24
3.7	Full verified example: FA at wB97x-D3/def2-TZVP	25
3.8	Common troubleshooting	25
3.9	What's next	26
4	High-level (HL) calculations with UMA (MLIP)	27
4.1	What changes with MLIP vs. ORCA	27
4.2	Requirements	27
4.3	Adjusting the <code>amk.dat</code>	27
4.4	Launching the calculation	28
4.5	GPU vs. CPU	28
4.6	Where results live	28
4.7	Full verified example: FA with UMA	28
4.8	Common troubleshooting	28
4.9	What's next	29
5	Visualization and analysis with <code>amk_tools</code>	30
5.1	What <code>amk_tools</code> is, and why it lives in a separate environment	30
5.2	Isolated installation	30
5.3	The two tools: overview	31
5.4	<code>amk_gen_view.py</code> : complete option reference	31
5.5	Practical example: network from the global MIN, DFT vs UMA	32
5.6	<code>amk_rxn_stats.py</code> : network statistics	33
5.7	Common troubleshooting	34
5.8	What's next	34

Introduction

This appendix is a from-zero, step-by-step walkthrough aimed at people who are **not yet familiar with AutoMeKin or computational chemistry**. It complements the rest of the Tutorial section above (which assumes some prior familiarity with the program) by spelling out every command, every file, and every error message a first-time user is likely to run into — all of it tested end to end on the same worked example (formic acid, **FA**) before being written down.

Note: This appendix is self-contained: you can follow it on its own without having read the other Tutorial pages first.

0.1 Contents

1. **Installation from zero** — system dependencies, Miniconda, building AutoMeKin from source, and setting up the optional high-level backends (ORCA, MACE, UMA).
2. **Low-level (LL) calculations** — preparing an input file, running `llcalcs.sh`, and reading the resulting reaction network.
3. **High-level (HL) calculations with ORCA** — refining the LL transition states with DFT.
4. **High-level (HL) calculations with UMA (MLIP)** — refining the same transition states with a machine-learning interatomic potential instead, and comparing cost vs. accuracy against DFT.
5. **Visualization and analysis with `amk_tools`** — generating interactive reaction-network visualizations and network statistics from your results.

Chapter 1

Installation from zero

This page is aimed at people who **don't know the subject yet**: no prior knowledge of computational chemistry is assumed, nor any experience compiling software from source. Every step includes the exact command and a short explanation of what it does and why.

Note: All commands on this page were tested on Ubuntu 22.04 LTS.

1.1 What is AutoMeKin?

AutoMeKin (formerly called **tsscads**) is an open-source program that automatically discovers chemical reaction mechanisms: starting from a single input molecular structure, it locates transition states using molecular dynamics simulations and graph theory algorithms, then computes reaction kinetics with Monte Carlo simulations.

You don't need to understand the chemistry to install it — just basic Linux terminal skills (copy-pasting commands, knowing which folder you're in).

1.2 Prerequisites

- **Operating system:** Debian/Ubuntu-based Linux (this page uses Ubuntu 22.04 LTS). Build scripts also exist for CentOS/RHEL, and there's an alternative Singularity (container) method if you can't compile on your own machine.
- **Disk space:** at least 3-4 GB free for the basic installation (source code + Python dependencies + build artifacts). If you're also going to use the high-level backends from Step 7 (ORCA and/or the MACE/UMA models), set aside **15-20 GB extra** — the UMA model alone is about 11 GB.
- **Administrator privileges (sudo)** to install system packages.
- **Internet connection** to download the code and dependencies.
- **NVIDIA GPU (optional):** not required, but it greatly speeds up calculations with the MACE/UMA models from Step 7. Without a GPU, those models still work, just slower (they run on CPU).
- Know how to open a terminal and run commands. No programming knowledge required.

1.3 Installation roadmap

Installing AutoMeKin from zero has several parts. It's longer than a simple `apt install` because AutoMeKin is **compiled** from its source code:

1. System packages (compilers, git, tools)

- ```

 v
2. Python environment (Miniconda + scientific libraries)
 |
 v
3. Download the source code (git clone)
 |
 v
4. Build and install (autoreconf \ensuremath{\rightarrow} configure
 ↪ \ensuremath{\rightarrow} make \ensuremath{\rightarrow} make install)
 |
 v
5. Configure the PATH so the "amk" commands work from any folder
 |
 v
6. (Optional) High-level backends: ORCA and/or MACE/UMA models

```

Step 7 (ORCA, MACE, UMA) is **only needed if you're going to run high-level (HL) calculations**. By default, AutoMeKin already ships with MOPAC for exploring reactions at low level (LL), so it can be installed without this extra step and added later whenever it's needed — without having to recompile anything.

## 1.4 Step 1 — System dependencies

AutoMeKin needs build tools (C, Fortran), shell utilities and an SQLite database.

**Optional: check what's already installed.** Many of these come preinstalled on Ubuntu, or may already be present from other projects. This just prints OK or MISSING for each one — nothing gets changed:

```

for pkg in git curl build-essential gfortran autoconf automake gawk bc \
 parallel sqlite3 libsqlite3-dev; do
 dpkg -s "$pkg" &>/dev/null && echo "OK $pkg" || echo "MISSING $pkg"
done

```

This step can be skipped — the install command below is safe to run regardless, since apt simply skips whatever is already installed:

```

sudo apt update
sudo apt install -y \
 git curl \
 build-essential gfortran \
 autoconf automake \
 gawk bc \
 parallel \
 sqlite3 libsqlite3-dev

```

What does each group install?

| Package(s)                | What it's for                                                      |
|---------------------------|--------------------------------------------------------------------|
| git, curl                 | Download the source code and auxiliary files                       |
| build-essential, gfortran | C/C++ and Fortran compilers (AutoMeKin has code in both languages) |
| autoconf, automake        | Generate the <b>configure</b> script from the source code          |
| gawk, bc                  | Text and calculation utilities used by internal scripts            |
| parallel                  | Runs several calculations in parallel (speeds up simulations)      |
| sqlite3, libsqlite3-dev   | Database where AutoMeKin stores intermediate results               |

## 1.5 Step 2 — Python with Miniconda

AutoMeKin uses several scientific Python libraries (ASE, NumPy, SciPy, NetworkX, Matplotlib). The cleanest way to install them without interfering with the system Python is to create a dedicated **conda environment**.

### 1.5.1 2.1 Install Miniconda (if not already present)

```
curl -O https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
```

Accept the license, and when prompted, let the installer initialize conda in the shell. Close and reopen the terminal (or run `source ~/.bashrc`) so the `conda` command becomes available.

**Note:** If Miniconda or Anaconda is already installed, this step can be skipped.

### 1.5.2 2.2 Create the environment for AutoMeKin

```
conda create -n amk_env python=3.11 -y
conda activate amk_env
```

**Note:** If this fails complaining about the Terms of Service for the `pkgs/main/pkgs/r` channels not being accepted, run these two commands (the error message itself says the same thing) and then retry `conda create`. This is a one-time Anaconda requirement, unrelated to AutoMeKin itself.

```
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
```

### 1.5.3 2.3 Install the required libraries

```
conda install -y -c conda-forge numpy scipy matplotlib networkx sqlite joblib
pip install ase
```

Verify everything installed correctly:

```
python3 -c "import ase, numpy, scipy, networkx, matplotlib, joblib; print('All good')"
```

It should print `All good` with no errors.

**Important:** Every time a new terminal is opened to build or run AutoMeKin, activate the environment first with `conda activate amk_env`.

## 1.6 Step 3 — Download the source code

With the `amk_env` environment activated, pick a folder for the project (for example, the home folder) and clone the repository:

```
mkdir -p ~/automekin
cd ~/automekin
git clone https://github.com/ComputationalChem-USC/AutoMeKin-X.git src
cd src
```

This creates `~/automekin/src` with all the source code.

## 1.7 Step 4 — Build and install AutoMeKin

This is the actual “compilation” step. It follows the standard GNU toolchain flow: `autoreconf` → `configure` → `make` → `make install`.

From `~/automekin/src`:

```
autoreconf -i
./configure --prefix=$HOME/automekin/install
make
make install
```

### What does each command do?

- `autoreconf -i` — generates the `configure` script from the project’s templates (also installs any missing auxiliary files).
- `./configure --prefix=...` — checks that all dependencies are present and prepares the build to install into the folder given by `--prefix` (here, `~/automekin/install`). **Check the summary it prints at the end:** if something is missing, it will say so explicitly.
- `make` — compiles the code (takes a few minutes).
- `make install` — copies the compiled binaries, scripts and modules to `~/automekin/install`.

If `make` fails because a package is missing, go back to Step 1, install what’s missing, and repeat `./configure` and `make` (no need to repeat `autoreconf`).

## 1.8 Step 5 — Set up the environment permanently

To be able to run AutoMeKin’s commands from any folder, in any new terminal, add the following to the end of `~/.bashrc`:

```
--- AutoMeKin ---
export AMK=$HOME/automekin/install
export PATH=$AMK/bin:$AMK/bin/HLscripts:$AMK/bin/MOPAC_DEV:$PATH
export LIBRARY_PATH=$AMK/lib:$AMK/bin/MOPAC_DEV
```

This can be added by editing the file directly, or with this command (careful: use `>>` to **append**, not `>`, which would erase the file):

```
cat >> ~/.bashrc << 'EOF'

--- AutoMeKin ---
export AMK=$HOME/automekin/install
export PATH=$AMK/bin:$AMK/bin/HLscripts:$AMK/bin/MOPAC_DEV:$PATH
export LIBRARY_PATH=$AMK/lib:$AMK/bin/MOPAC_DEV
EOF
```

Then apply the changes:

```
source ~/.bashrc
```

**Note:** Since `amk_env` is also in use, remember that both `conda activate amk_env` (for the Python libraries) and this `~/.bashrc` block (to find commands like `amk.sh`) are needed in every new terminal. To have the conda environment activate automatically, `conda activate amk_env` can also be added to the end of `~/.bashrc`, though it’s not required.

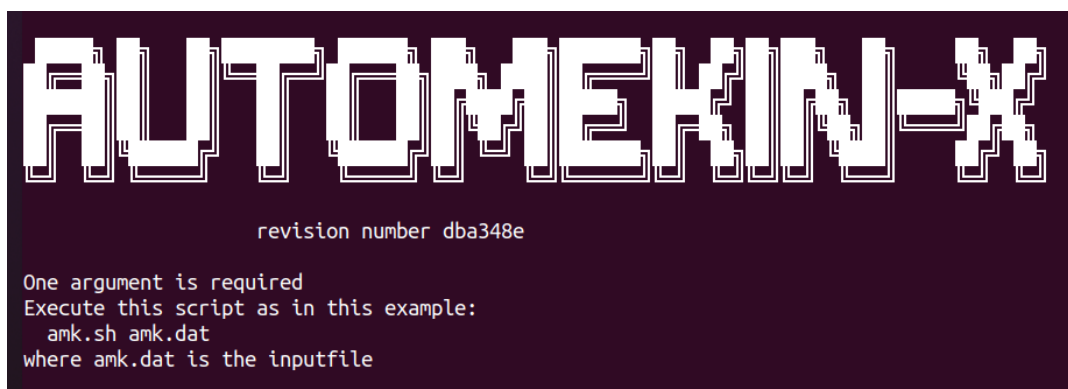


Figure 1.1: AutoMeKin ASCII logo banner

## 1.9 Step 6 — Verify that everything works

Open a new terminal (so the `.bashrc` changes take effect), activate the environment, and check that the main command responds:

```
conda activate amk_env
amk.sh
```

If everything is installed correctly, AutoMeKin’s ASCII logo appears, followed by a message like this:

```
One argument is required
Execute this script as in this example:
 amk.sh amk.dat
where amk.dat is the inputfile
```

This message **confirms the installation worked**: the program runs, recognizes its dependencies, and is only complaining that no input file was given (covered in the next page).

As an extra check, AutoMeKin ships with a test suite that runs a real case end to end:

```
cd $AMK/../../src # or the folder where the repo was cloned
run_test.sh --tests=FA
```

FA is the reference example (formic acid) used to validate that the whole calculation chain works correctly. At this point, before Step 7 below, the results obtained will be **low-level (LL) only**.

## 1.10 Step 7 — (Optional) High-level backends: ORCA, MACE and UMA

### 1.10.1 7.1 What are LL and HL, and when is this step needed?

AutoMeKin works at two levels:

- **LL (Low Level)**: fast but less accurate. Used to explore a huge number of possible reactions. It uses **MOPAC** by default, which is already included — nothing extra needs to be installed for this.
- **HL (High Level)**: more accurate but much more expensive. Used to refine only the most promising results found at LL.

If the input file (covered on the next page) is going to use the `HighLevel` keyword with something other than MOPAC, the corresponding engine needs to be installed. AutoMeKin supports several

backends for HL:

| Backend                                      | HighLevel keyword          | What needs to be installed                                                                                                                                                                 |
|----------------------------------------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gaussian 09/16<br><b>ORCA</b><br>Entos Qcore | g09 / g16<br>orca<br>qcore | A Gaussian license (not covered here)<br>Free download for academic use (8.2)<br>conda package <code>qcore</code> (see the repository's <code>automekin.yml</code> for a specific version) |
| <b>MLIP: MACE</b>                            | mlip mace                  | Python libraries + the <code>MACE-omol-0-extra-large-1024.model</code> model (7.3)                                                                                                         |
| <b>MLIP: UMA</b>                             | mlip uma                   | Python libraries + the <code>uma-m-1p1.pt</code> model (7.3)                                                                                                                               |

This section covers **ORCA** and the **MLIP models (MACE and UMA)**, the most common, freely/openly accessible options.

**Note:** Only one of the three needs to be installed (e.g. only ORCA, or only MACE) — there's no need to install all of them unless all are going to be used.

### 1.10.2 7.2 ORCA

**ORCA** is a free-for-academic-use quantum chemistry program, but **it's not a simple curl or apt download**: it requires registering on its official forum and accepting the license.

#### 1. Create an account and request the download

- Register at <https://orcaforum.kofo.mpg.de/index.php?register/>.
- Confirm the email and accept the EULA (license agreement).
- Log in and go to the *Filebase* section. Choose the **Linux x86-64** build, “**shared, OpenMPI**” variant (matching the target distribution; on Ubuntu 22.04 the build against OpenMPI 4.1 is used), and download the `.tar.xz` archive.

#### 2. Install OpenMPI 4.1 (a required ORCA dependency)

ORCA doesn't bundle OpenMPI — it needs to find it on the `PATH`.

```
sudo apt install -y openmpi-bin libopenmpi-dev
mpirun --version
```

#### 3. Extract ORCA

The downloaded `.tar.xz` (typically in the Downloads folder) can be extracted wherever the program should live, for example:

```
mkdir -p ~/orca6
tar xf ~/Downloads/orca_6*_linux_x86-64_shared_openmpi*.tar.xz -C ~/orca6
```

#### 4. Add it to the PATH

```
cat >> ~/.bashrc << 'EOF'

--- ORCA ---
export PATH="$HOME/orca6/orca6/bin:$PATH"
EOF
source ~/.bashrc
```

(Adjust the `~/orca6/orca6` path to whatever folder name was actually created when extracting the tarball — it usually includes the version number.)

 ORCA FORUM

## Registration

Existing user? [Login](#)

Username \*

Username must be 3 up to 25 characters long.

Email Address \*

Password \*



Password Strength

A secure password should be at least 10 characters long.

User Interface Language

 English ▼

☐ I agree to the terms and conditions. \*

Verification ("reCAPTCHA")

☐ No soy un robot

  
reCAPTCHA

Register

Figure 1.2: ORCA forum registration page

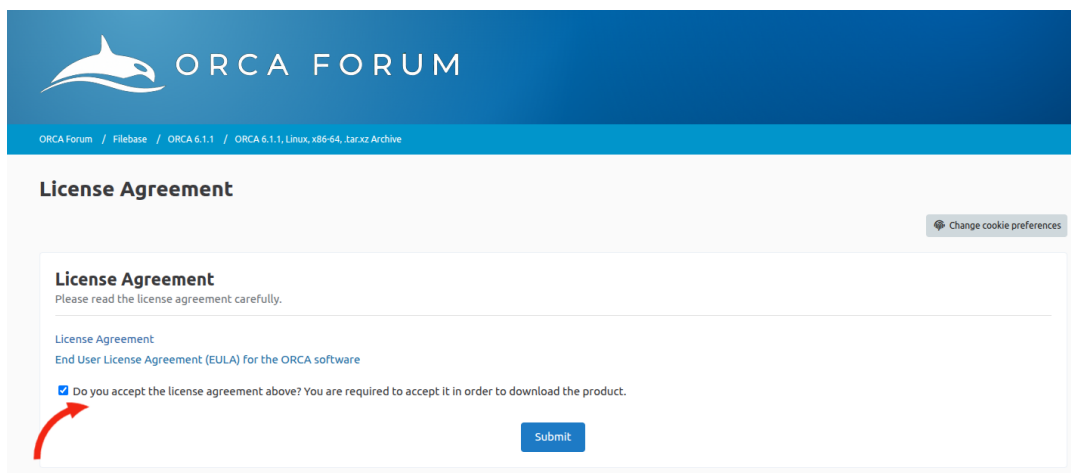


Figure 1.3: ORCA EULA agreement

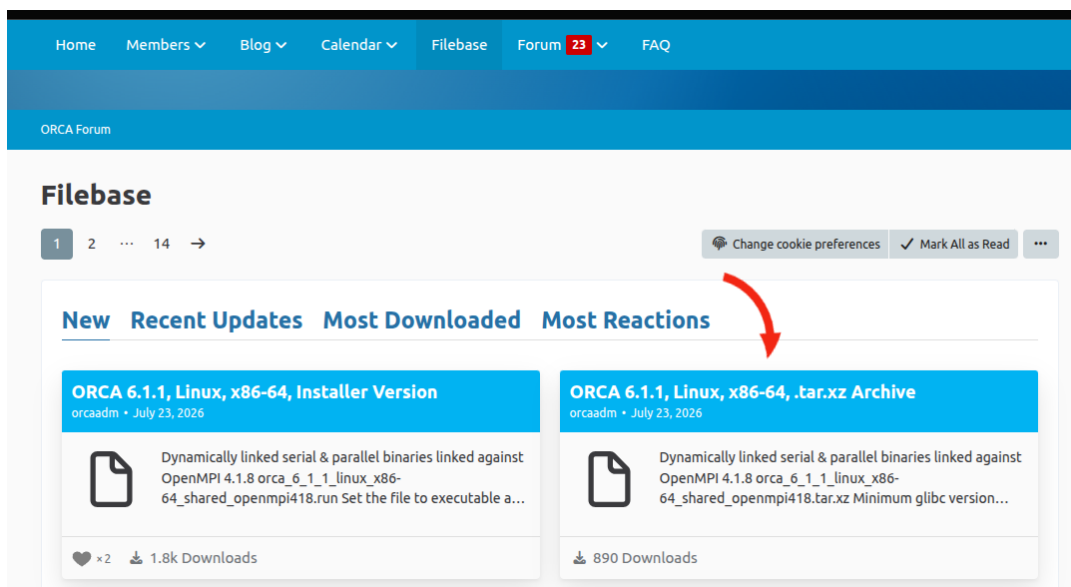


Figure 1.4: ORCA Filebase download section

## 5. Verify

```
orca
```

This should print the ORCA banner. If `orca: command not found` appears instead, check that the `PATH` entry points to the folder that actually contains the `orca` executable. Depending on the ORCA version, this may or may not be inside a `bin/` subfolder — some versions install the executable directly in the extracted folder, in which case the `PATH` entry should point there instead, e.g. `export PATH="$HOME/orca6/orca6:$PATH"`.

### 1.10.3 7.3 Machine-learning interatomic potentials (MLIP): MACE and UMA

MACE and UMA are **machine-learning models** that replace a traditional quantum chemistry program for the HL step. With them, Gaussian or ORCA are not needed: AutoMeKin calls the model directly.

#### 1. Install the required Python libraries

With the `amk_env` environment activated (`conda activate amk_env`), install them in **two separate pip install commands, in this exact order** — not all in one command:

```
pip install torch sella omegaconf fairchem-core
pip install mace-torch

PyTorch >=2.6 needs this to load a file e3nn ships with itself
(harmless to run even if the e3nn build doesn't need it -- it's a no-op then)
E3NN_WIGNER=$(python3 -c "import e3nn, os;
→ print(os.path.join(os.path.dirname(e3nn.__file__), 'o3', '_wigner.py'))")
sed -i "s/'constants.pt')/'constants.pt'), weights_only=False)/" "$E3NN_WIGNER"

Avoids a JIT-compile crash on older g++ when UMA first runs
(also harmless if the compiler is fine -- just skips that speed optimization)
echo 'export TORCHDYNAMO_DISABLE=1' >> ~/.bashrc
export TORCHDYNAMO_DISABLE=1
```

**Note: Why two commands?** `mace-torch` requires an exact version, `e3nn==0.4.4`, while `fairchem-core` requires `e3nn>=0.5`. Those two constraints can't both be satisfied at once, so asking `pip` to install all five packages together in a single command fails with a `ResolutionImpossible` error. Installing `fairchem-core` first and `mace-torch` second sidesteps this: `pip` installs `e3nn>=0.5` for `fairchem-core`, then the second command quietly downgrades it to `e3nn==0.4.4` for `mace-torch`. A warning like `fairchem-core 2.20.0 requires e3nn>=0.5, but you have e3nn 0.4.4 which is incompatible` is expected at the end of the second command, and is safe to ignore — AutoMeKin only uses the parts of `fairchem-core` that work fine with `e3nn 0.4.4`.

If an NVIDIA GPU is available, install the CUDA-enabled `torch` build matching the driver first, following the instructions at [pytorch.org](https://pytorch.org), then run the two commands above (they'll reuse the `torch` already installed). Without a GPU, the standard `pip install torch` works fine too, it just runs on CPU.

#### 2. Create the models folder

AutoMeKin looks for models in a fixed path inside the installation, `$AMK/models/` (this is not configurable from the input file):

```
mkdir -p $AMK/models
```

### 3. Download the MACE model

The `MACE-omol-0-extra-large-1024.model` model (~400 MB, ASL academic license) can be downloaded directly, with no account required, from the GitHub releases:

```
wget -O $AMK/models/MACE-omol-0-extra-large-1024.model \
 https://github.com/ACESuit/mace-foundations/releases/download/mace_omol_0/MACE-omol-0_
↪ -extra-large-1024.model
```

### 4. Download the UMA model

The `uma-m-1p1.pt` model (~11 GB) is hosted on Hugging Face and **requires accepting a license** before it can be downloaded:

- Create an account at <https://huggingface.co/join> if needed.
- Go to <https://huggingface.co/facebook/UMA> and accept the “FAIR Chemistry License v1” (it will ask for a full legal name, date of birth, and organization).

**Note:** Approval for the request usually takes a few minutes. Once accepted, an email granting access to the UMA models arrives, like this one:

- Generate an access token at <https://huggingface.co/settings/tokens> (a read-only one is enough).

Once the account is approved, install the client and authenticate:

```
pip install -U huggingface_hub
hf auth login
Paste the token here when prompted
```

Download the model, then move it out of the `checkpoints/` subfolder that `--local-dir` recreates (same reason as the `atom-refs` file below — `--local-dir` preserves the repository’s internal folder structure):

```
hf download facebook/UMA checkpoints/uma-m-1p1.pt \
 --local-dir $AMK/models
mv $AMK/models/checkpoints/uma-m-1p1.pt $AMK/models/uma-m-1p1.pt
rmdir $AMK/models/checkpoints
```

(Use `huggingface-cli download ...` instead of `hf download ...` on an older `huggingface_hub` version without the `hf` command.)

This can take a while given the file size (11 GB).

The optional atom-reference file — recommended because it improves accuracy for single-atom fragments — lives in the same repository under `references/iso_atom_elem_refs.yaml`, not at the repository root, and under a different name than the one AutoMeKin expects. Download it and rename it in one go:

```
hf download facebook/UMA references/iso_atom_elem_refs.yaml \
 --local-dir $AMK/models
mv $AMK/models/references/iso_atom_elem_refs.yaml \
 $AMK/models/uma-m-1p1_atom_refs.yaml
rmdir $AMK/models/references
```

`--local-dir` preserves the repository’s folder structure, so the file first lands at `$AMK/models/references/iso_atom_elem_refs.yaml` — the `mv` puts it directly in `$AMK/models/` under the exact name AutoMeKin looks for.



# Join Hugging Face

Join the community of machine learners!

Email Address

Hint: Use your organization email to easily find and join your company/team org.

Password

Next

Already have an account? [Log in](#)

[Unlock SSO](#) with  Team & Enterprise

Figure 1.5: Hugging Face login page

facebook/UMA like 365 Follow AI at Meta 14.5k

arxiv:2506.23971 License: other

Model card Files and versions xet

**You need to agree to share your contact information to access this model**

This repository is publicly accessible, but you have to accept the conditions to access its files and content.

Please be sure to provide your full legal name, date of birth, and full organization name with all corporate identifiers. Avoid the use of acronyms and special characters. Failure to follow these instructions may prevent you from accessing this model and others on Hugging Face. You will not have the ability to edit this form after submission, so please ensure all information is accurate.

**FAIR Chemistry License v1 Last Updated: May 14, 2025**

"Acceptable Use Policy" means the FAIR Chemistry Acceptable Use Policy that is incorporated into this Agreement.

"Agreement" means the terms and conditions for use, reproduction, distribution and modification of the Materials set forth herein....

[Log in](#) or [Sign Up](#) to review the conditions and access this model content.

## UMA: A Family of Universal Models for Atoms

Figure 1.6: FAIR Chemistry License agreement

Hi [redacted]

This is to let you know your request to access model "facebook/UMA" on huggingface.co has been accepted by the repo authors.

You can now access the repo [here](#), or view all your gated repo access requests [in your settings](#).

Cheers,

The Hugging Face team

Visual Watermark

Figure 1.7: UMA access approval email

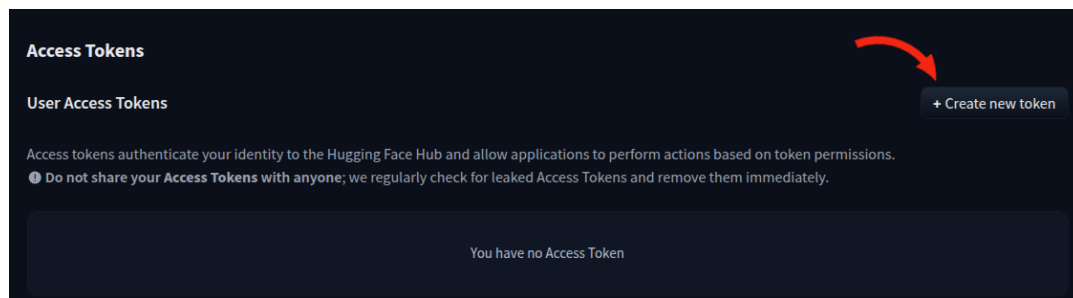


Figure 1.8: Hugging Face access token generation

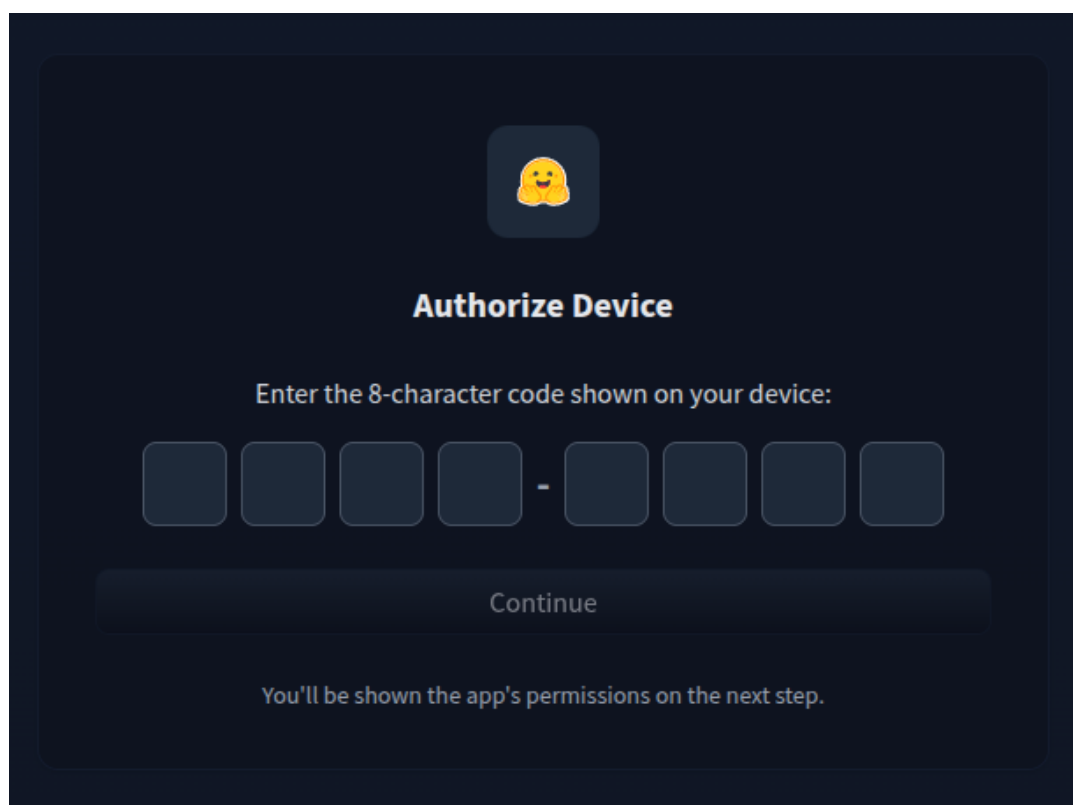


Figure 1.9: Hugging Face CLI login

**Warning:** AutoMeKin looks for models by their **exact file name** (MACE-omol-0-extra-large-1024.model, uma-m-1p1.pt, uma-m-1p1\_atom\_refs.yaml). If the name doesn't match, the calculation fails with an error like `MLIP model file not found: ...`. The commands above already save the file with the correct name; when downloading manually from a browser, double-check it wasn't saved with a suffix like (1).

## 5. Verify where the models ended up

```
ls -lh $AMK/models
```

The `.model` / `.pt` / `.yaml` files downloaded above should appear.

With this in place, the input file can use `HighLevel mlip mace` or `HighLevel mlip uma`.

## 1.11 Common troubleshooting

| Symptom                                                                                               | Likely cause                                                                                                                                                               | Fix                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>amk.sh: command not found</code>                                                                | The Step 5 block isn't in <code>~/.bashrc</code> , or a new terminal wasn't opened                                                                                         | Check <code>~/.bashrc</code> and run <code>source ~/.bashrc</code>                                                                         |
| <code>configure: error: ...</code> mentioning a package<br><code>ModuleNotFoundError</code> in Python | A system dependency is missing<br><br>The conda environment isn't activated, or a library is missing                                                                       | Go back to Step 1 and install the package it names<br><code>conda activate amk_env</code> and redo Step 2.3                                |
| <code>make</code> fails partway through                                                               | Usually an incomplete <code>configure</code>                                                                                                                               | Delete the <code>src</code> folder, clone it again, and redo Step 4 from scratch                                                           |
| Permission denied installing packages with <code>apt</code>                                           | Missing <code>sudo</code>                                                                                                                                                  | Add <code>sudo</code> before the <code>apt install</code> command                                                                          |
| <code>orca: command not found</code>                                                                  | ORCA's <code>bin</code> folder isn't on the <code>PATH</code> , or a new terminal wasn't opened                                                                            | Check the <code># --- ORCA ---</code> block in <code>~/.bashrc</code> and run <code>source ~/.bashrc</code>                                |
| ORCA hangs or won't run in parallel                                                                   | Missing OpenMPI 4.1, or a version mismatch                                                                                                                                 | <code>sudo apt install -y openmpi-bin libopenmpi-dev</code> and confirm with <code>mpirun --version</code>                                 |
| <code>huggingface-cli</code> says "deprecated and no longer works"                                    | <code>huggingface_hub</code> 1.0+ is installed, which renamed the CLI                                                                                                      | Use <code>hf auth login</code> / <code>hf download ...</code> instead                                                                      |
| 403 Forbidden downloading the UMA model                                                               | The license on the model page hasn't been accepted, or authentication ( <code>hf auth login</code> ) is missing                                                            | Go to <a href="https://huggingface.co/facebook/UMA">https://huggingface.co/facebook/UMA</a> , accept the license, and re-authenticate      |
| <code>pip</code> fails with <code>ResolutionImpossible</code> mentioning <code>e3nn</code>            | <code>mace-torch</code> and <code>fairchem-core</code> were installed in the same <code>pip install</code> command — their <code>e3nn</code> version requirements conflict | Install them separately: <code>pip install torch sella omegaconf fairchem-core</code> , then <code>pip install mace-torch</code> (see 7.3) |
| <code>MLIP model file not found: ...</code> when launching a run                                      | The model isn't in <code>\$AMK/models/</code> , or the file name doesn't match exactly                                                                                     | Run <code>ls -lh \$AMK/models</code> and compare the name against the table in 7.3                                                         |
| UMA download is very slow or gets interrupted                                                         | 11 GB file, sensitive to unstable connections                                                                                                                              | Re-run the same <code>hf download</code> (or <code>huggingface-cli download</code> ) command: it resumes instead of starting over          |

## 1.12 What's next

With this, AutoMeKin is installed and verified. The next page covers how to actually use it, from zero: preparing a starting molecular structure, writing an input file, launching a first real calculation, and reading the results.

## Chapter 2

# Low-level (LL) calculations

### 2.1 LL vs HL: what this page covers

As covered on the installation page, AutoMeKin works at two levels:

- **LL (Low Level):** fast, uses MOPAC by default. Hundreds of molecular dynamics trajectories are launched from the starting structure to **discover** candidate transition states (TS), without worrying yet about final accuracy.
- **HL (High Level):** slow and accurate, supporting several computational chemistry programs (Gaussian, ORCA, Qcore, or an MLIP model). Applied only **afterward**, to the most promising TSs already found at LL — covered on the next two pages.

This page stays entirely at **LL**: AutoMeKin is used to explore a molecule's potential energy surface with MOPAC and build its first reaction network.

### 2.2 Setting up the working folder

Create a folder for this exercise and enter it:

```
mkdir -p test_example_dft
cd test_example_dft
```

Two files are needed: the starting structure (**.xyz**) and the input file (**.dat**).

#### 2.2.1 Starting structure

This example uses formic acid ( $\text{CH}_2\text{O}_2$ ) — it's the reference molecule AutoMeKin uses in its own test suite. Save this as **FA.xyz**:

```
5

C 0.000000 0.000000 0.000000
O 0.000000 0.000000 1.220000
O 1.212436 0.000000 -0.700000
H -0.943102 0.000000 -0.544500
H 1.038843 0.000000 -1.634005
```

The first line is the atom count, the second is left blank (or holds a comment), then one line per atom: symbol + X Y Z coordinates in Å. It doesn't need to be a perfectly optimized geometry — MOPAC optimizes it as a first step anyway.

## 2.3 The amk.dat input file

AutoMeKin needs an input file with the instructions for the calculations to run: which molecule to use, which method to explore the potential energy surface with (LL), which method to refine the results with (HL), and other settings that control the search.

Save this as FA.dat in the same folder:

```
--General--
molecule FA
LowLevel mopac pm7 t=5m
HighLevel orca wB97x-D3/def2-TZVP
HL_rxn_network complete
IRCpoints 29
charge 0
mult 1
timeout 1000

--Method--
sampling MD
ntraj 10
barrierless yes

--Screening--
imagmin 200
MAPEmax 0.008
BAPEmax 2.5
eigLmax 0.1

--Kinetics--
Energy 150
```

By section:

| Section       | Keyword                            | Meaning                                                                                                                         |
|---------------|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| --General--   | molecule FA                        | Must match FA.xyz                                                                                                               |
|               | LowLevel mopac pm7 t=5m            | LL engine: MOPAC, PM7 method.<br>t=5m is MOPAC's own keyword<br>(internal 5-minute limit per<br>calculation)                    |
|               | HighLevel orca ...                 | <b>Required even though this page<br/>doesn't use it</b> — see the warning<br>below                                             |
|               | charge, mult<br>timeout 1000       | Charge and spin multiplicity<br>Max seconds a single task may run<br>before it's killed (protects against<br>hung calculations) |
| --Method--    | sampling MD                        | Molecular-dynamics sampling (the<br>default method)                                                                             |
|               | ntraj 10                           | Internal trajectory-sampling<br>parameter                                                                                       |
|               | barrierless yes                    | Also search for barrierless dissociation<br>channels (NEB)                                                                      |
| --Screening-- | imagmin, MAPEmax, BAPEmax, eigLmax | Thresholds for filtering real TS<br>candidates                                                                                  |
| --Kinetics--  | Energy 150                         | Internal energy (kcal/mol) for the<br>Monte Carlo kinetics simulation                                                           |

**Warning:** Even for a 100% LL calculation, the HighLevel keyword is required — omitting it causes an error (HighLevel keyword has not been defined).

## 2.4 Launching the calculation

The script that orchestrates the whole LL search, iteration after iteration, is `llcalcs.sh`. Besides the input file, it takes three arguments — `ntasks`, `niter`, and `runningtasks`:

| Position              | Name                   | Meaning                                                           |
|-----------------------|------------------------|-------------------------------------------------------------------|
| 1st ( <b>FA.dat</b> ) | input file             |                                                                   |
| 2nd (10)              | <b>ntasks</b> (nbatch) | How many new tasks/trajectories get launched <b>per iteration</b> |
| 3rd (5)               | <b>niter</b>           | Maximum number of search-loop iterations                          |
| 4th (10)              | <b>runningtasks</b>    | How many tasks run <b>concurrently</b> at once                    |

10 5 10 is used in this example because formic acid is a small, fast-to-compute molecule: 10 trajectories per iteration over 5 iterations (50 trajectories total) is enough to sample its potential energy surface thoroughly, and running all 10 concurrently keeps the whole search quick on a normal desktop.

It's invoked like this:

```
nohup llcalcs.sh FA.dat 10 5 10 > llcalcs.log 2>&1 &
```

With `ntasks=10` and `runningtasks=10`, all 10 tasks of each iteration start at once (on a machine with  $\geq 10$  free cores). If `runningtasks` were smaller than `ntasks`, they'd run in batches instead.

`nohup ... &` sends the process to the background and keeps it running even if the terminal is closed. Monitor it with:

```
tail -f llcalcs.log
```

## 2.5 What happens while it runs: the iteration loop

Each loop iteration does, in order:

1. **Running TS search** — launches `ntasks` new MD trajectories in parallel (up to `runningtasks` at a time), looking for candidate transition states.
2. **Running IRC** — for each TS candidate, integrates the reaction path forward and backward (IRC) to confirm which minima it connects.
3. **Checks for new TSs**. If none were found this iteration, it's discarded and the loop continues. If this happens **3 iterations in a row**, the loop stops early due to convergence, even if `niter` hasn't been reached yet.
4. **Running min opt** — optimizes the minima connected to the new TSs.
5. **Building network** — rebuilds the full reaction network with everything found so far.
6. **Running Kinetics** — recomputes the Monte Carlo kinetics with the updated network.

Once all iterations finish (or convergence stops it early), if `barrierless yes` was set, one more phase runs:

7. **Adding Barrierless reactions** — searches for barrierless dissociation channels with NEB.

Then the final results folder is built.

## 2.6 Where results actually live — and why `batch*` disappears

There are two places where data actually persists:

### 2.6.1 tsdirLL\_<molecule>/ — the working database (accumulates iteration after iteration)

| Folder                | Contents                                                                                     |
|-----------------------|----------------------------------------------------------------------------------------------|
| TSs/                  | MOPAC outputs for every TS found + <code>ts.db</code>                                        |
| IRC/                  | IRC integrations (forward/backward) from each TS                                             |
| MINs/                 | Optimized minima + <code>min.db</code>                                                       |
| PRODs/                | Identified products + <code>prod.db</code>                                                   |
| KMC/                  | Kinetics results: <code>RXNet</code> , <code>kmcE*.out</code> , <code>branchingE*.out</code> |
| <code>track.db</code> | History of TSs per iteration (what the script uses to decide convergence)                    |

This folder can be inspected at any time, even while the run is still active, to see the raw detail of a specific TS or minimum.

**Note:** `batch1`, `batch2`, ... folders appear in the working directory while a run is active — those are the *scratch* space for each individual task. `llcalcs.sh` deletes them automatically once their useful results have been copied elsewhere. Not seeing them after a run finishes is expected, not a problem.

### 2.6.2 FINAL\_LL\_<molecule>/ — the curated final summary

Generated once, when the whole loop finishes. This is the folder to read results from — covered next.

## 2.7 Reading the final results

Inside `FINAL_LL_FA/` (replace `FA` with the molecule's name):

| File                                                            | What it is                                                                                                                                                                                                            |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>RXNet</code>                                              | The full reaction network: each TS, its relative energy ( $\Delta E$ in kcal/mol vs. the global minimum), and what it connects (MIN i <---> MIN j, or MIN i <---> PRk: formula for irreversible channels to products) |
| <code>RXNet.rel</code> / <code>RXNet.cg</code>                  | Filtered/reduced variants of the network (only the relevant channels, or the connected graph)                                                                                                                         |
| <code>RXNet.barrless</code>                                     | Barrierless dissociation channels found by NEB                                                                                                                                                                        |
| <code>MINinfo</code>                                            | List of minima with their $\Delta E$ ; conformational isomers are grouped on the same line                                                                                                                            |
| <code>TSinfo</code>                                             | List of TSs with their $\Delta E$ (same format)                                                                                                                                                                       |
| <code>convergence.txt</code>                                    | How many TSs had been found after each batch of 100 trajectories — useful to see whether sampling is flattening out (converging) or whether more iterations would help                                                |
| <code>kinetics.csv</code>                                       | Time evolution (Monte Carlo) of each species' population                                                                                                                                                              |
| <code>rxn_kin.txt</code>                                        | Branching ratios: what fraction of the reaction flux ends up at each product/minimum from the starting point                                                                                                          |
| <code>min.db</code> , <code>ts.db</code> , <code>prod.db</code> | The same SQLite databases, now consolidated                                                                                                                                                                           |

## 2.8 Full verified example: FA (formic acid)

With the exact setup above (`ntasks=10 niter=5 runningtasks=10`), this was the result of a complete run:

**convergence.txt** — sampling flattened out ( $7 \rightarrow 11 \rightarrow 12 \rightarrow 12 \rightarrow 13$  TSs every 100 more trajectories):

| Iter # | TSs | ntrajs |
|--------|-----|--------|
| 1      | 7   | 100    |
| 2      | 11  | 200    |
| 3      | 12  | 300    |
| 4      | 12  | 400    |

**RXNet** — 13 TSs found, including the two known decomposition channels of formic acid:

| TS # | DE(kcal/mol) | Reaction path information |               |
|------|--------------|---------------------------|---------------|
| ==== | =====        | =====                     | =====         |
| 1    | -1.4         | PR1: CO + H2O <--->       | PR1: CO + H2O |
| 2    | 1.9          | MIN 1 <--->               | MIN 2         |
| 6    | 32.8         | PR3: H2 + CO2 <--->       | PR1: CO + H2O |
| 7    | 37.6         | MIN 4 <--->               | PR1: CO + H2O |
| 9    | 44.0         | MIN 3 <--->               | PR3: H2 + CO2 |
| ...  |              |                           |               |

**rxn\_kin.txt** — 83.8% of the reaction flux from the most stable minimum ends up at CO + H<sub>2</sub>O, 1.8% at H<sub>2</sub> + CO<sub>2</sub>, and the rest gets trapped in intermediate isomers:

```
1 3 0.0854545
1 6 0.00545455
3 CO+H2O 0.0518182
1 CO+H2O 0.838182
3 H2+CO2 0.000909091
1 H2+CO2 0.0181818
```

**Note:** Slightly different numbers from a different run are expected: MD sampling starts from random temperatures/velocities, so each run explores a somewhat different set of trajectories.

## 2.9 Common troubleshooting

| Symptom                                                              | Cause                                                                   | Fix                                                                               |
|----------------------------------------------------------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| HighLevel keyword has not been defined                               | Missing HighLevel line in the .dat, even for an LL-only run             | Add HighLevel orca <method> (or g09/g16/qcore/mlip) — see the input file section  |
| HighLevel value is X, and it should be qcore, g09, g16, orca or mlip | An unsupported value (e.g. mopac) was put in HighLevel                  | Use one of the five valid values                                                  |
| No batch* folders after running                                      | They auto-delete when each iteration/the run finishes — normal behavior | Check tsdirLL_<molecule>/ for raw detail, or FINAL_LL_<molecule>/ for the summary |
| \$inputfile is not in this folder                                    | llcalcs.sh was run from a different directory                           | cd into the folder with the .dat and .xyz before launching                        |
| Number of batches and/or number of iterations have not been set      | llcalcs.sh was called with fewer than 4 arguments (outside SLURM)       | Use the full form: llcalcs.sh inputfile ntasks niter runningtasks                 |
| The run stops before reaching niter                                  | Convergence: 3 iterations in a row with no new TSs                      | Expected behavior, not an error — check convergence.txt                           |

## 2.10 What's next

With a complete LL reaction network in hand, the next page takes these same results and refines the most relevant TSs at **high level (HL)** with ORCA, comparing how the energies shift relative to LL.

## Chapter 3

# High-level (HL) calculations with ORCA

Continuation of the low-level calculations page: this page refines with ORCA (DFT) the reaction network discovered at low level with MOPAC.

### 3.1 What the HL step does

Low level (previous page) is cheap but imprecise: MOPAC/PM7 is good for **discovering** the topology of the reaction network (how many TSs there are, which minima they connect), but its energies aren't reliable. The **high-level (HL)** step takes exactly those already-located TSs and:

1. **Re-optimizes** them with a more accurate method (here, wB97x-D3/def2-TZVP DFT with ORCA).
2. Confirms with **frequencies** that they're still real saddle points.
3. Repeats the **IRC** to confirm which minima each one connects, now at this level of theory.
4. Rebuilds the reaction network and kinetics with the corrected energies.

HL **doesn't search for new TSs** — it only refines the ones already found. That's why there's no `ntasks/niter` like at LL: it's a single pass, not an exploration loop.

### 3.2 Requirements before starting

- The low-level page completed: `tsdirLL_<molecule>/` with its `tslist` already generated.
- ORCA installed and on the PATH (see the installation page, section 7.2).
- The same `amk.dat` from the LL page, with the `HighLevel` keyword already set — in this example `HighLevel orca wB97x-D3/def2-TZVP`.

**Warning:** One important adjustment to `amk.dat` before launching HL: if `timeout 1000` was set on the LL page for the MOPAC tasks, remove it (or raise it a lot) before this step. A DFT calculation with ORCA can easily take longer than 1000 seconds, and `parallel --timeout` would kill the task mid-calculation. Without the keyword, it falls back to the default (~11.6 days, effectively unlimited).

```
sed -i '/^timeout /d' FA.dat
```

### 3.3 Launching the calculation

The script that orchestrates the whole HL refinement is `hlcalcs.sh`. Unlike `llcalcs.sh`, it only takes **two arguments**:

```
nohup hlcalcs.sh FA.dat 10 > hlcalcs.log 2>&1 &
```

| Position     | Name                | Meaning                             |
|--------------|---------------------|-------------------------------------|
| 1st (FA.dat) | input file          | Same one from the LL page           |
| 2nd (10)     | <b>runningtasks</b> | How many ORCA jobs run concurrently |

**Important detail about parallelism:** outside SLURM, each ORCA job defaults to **1 single core** (`nprocs=${SLURM_CPUS_PER_TASK:-1}`, and outside SLURM that variable doesn't exist  $\rightarrow$  1). In other words, parallelism doesn't come from splitting cores within a single ORCA calculation, but from how many ORCA calculations run *at once* — which is exactly what **runningtasks** controls. With 10, up to 10 simultaneous **orca** processes run, each on 1 core.

Monitor it the same way as on the LL page:

```
tail -f hlcalcs.log
```

### 3.4 What happens while it runs: `hlcalcs.sh`'s phases

`hlcalcs.sh` runs, in order, a different script per phase:

1. **Running TS opt** — reads the TSs from the LL page and launches an ORCA TS optimization for each one (`OptTS TightOpt TightSCF Freq`), plus an optimization of the starting minimum. All run in parallel, **runningtasks** at a time.
2. **Running IRC** — first checks, using each optimized TS's imaginary frequency, which ones are still real saddle points; discards the ones that aren't. Of those remaining, it also excludes any that exceed the energy threshold set by **Energy** in `--Kinetics--`. For the rest, it launches forward and backward IRC.
3. **Running min opt** — optimizes the minima each IRC endpoint connects to.
4. **Building network** — rebuilds the full network with HL energies.
5. **Running kinetics** — recomputes the Monte Carlo kinetics with the corrected network.
6. **Adding barrless procs** — repeats the barrierless-channel search (NEB), this time at DFT level (only if **barrierless yes** was set).
7. **Running frags opt** — optimizes the final products.

Finally, the results folder is built.

### 3.5 Where results live

Exactly the same pattern as at LL, but with HL instead of LL in the names:

- **tsdirHL\_<molecule>/** — the persistent working database: TSs/, IRC/, MINs/, PRODs/, KMC/, with the actual ORCA `.inp/.out` files and the corresponding SQLite databases.
- **FINAL\_HL\_<molecule>/** — the curated final summary, generated once everything finishes.

### 3.6 Reading the final results

`FINAL_HL_<molecule>/` has the same files already familiar from LL: `RXNet`, `RXNet.rel`, `RXNet.cg`, `RXNet.barrless`, `MINinfo`, `TSinfo`, `kinetics.csv`, `rxn_kin.txt`, `min.db`, `ts.db`, `prod.db`. The

only difference is that the energies (DE(kcal/mol)) now come from DFT instead of a semiempirical (SQM) method — much more reliable, but also much more expensive to compute.

**Note:** There's no `convergence.txt` at HL — that file is specific to the LL iterative loop; HL is a single pass, so it doesn't apply.

### 3.7 Full verified example: FA at wB97x-D3/def2-TZVP

With `runningtasks=10` over the 13 TSs from the LL page, this was the result:

**Total time** (10 ORCA jobs in parallel, 1 core each): **~1h50min**.

**RXNet** — of the 13 candidate TSs from LL, **11 survived** HL refinement (2 didn't converge to a real saddle point at this level, or fell outside the energy threshold):

| TS #  | DE(kcal/mol) | Reaction path information |         |      |          |
|-------|--------------|---------------------------|---------|------|----------|
| ===== | =====        | =====                     |         |      |          |
| 1     | 7.6          | MIN                       | 1 <---> | MIN  | 2        |
| 2     | 29.4         | MIN                       | 1 <---> | MIN  | 1        |
| 5     | 64.1         | MIN                       | 2 <---> | PR2: | CO2 + H2 |
| 6     | 65.7         | MIN                       | 1 <---> | PR1: | CO + H2O |
| 9     | 73.0         | MIN                       | 4 <---> | PR1: | CO + H2O |
| 10    | 77.1         | MIN                       | 5 <---> | PR2: | CO2 + H2 |
| 11    | 122.2        | MIN                       | 6 <---> | MIN  | 6        |

**rxn\_kin.txt** — branching shifts once energies are corrected:

```
1 3 0.15326
1 CO2+H2 0.0736664
1 CO+H2O 0.724809
3 CO+H2O 0.042337
3 CO2+H2 0.00592718
```

72.5% of the flux ends up at CO + H<sub>2</sub>O, 7.4% at CO<sub>2</sub> + H<sub>2</sub>, and the rest gets trapped in the MIN 3 isomer.

### 3.8 Common troubleshooting

| Symptom                                                          | Cause                                                                                                                                 | Fix                                                                                                                            |
|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| orca does not seem to be installed / Aborting...                 | ORCA isn't on the PATH of the session launching the calculation                                                                       | Check the installation page, section 7.2; confirm with a bare <code>orca</code> that it prints the banner                      |
| Keyword <code>HL_rxn_network</code> has not been specified       | That line is missing from the <code>.dat</code>                                                                                       | Add <code>HL_rxn_network complete</code> (or <code>reduced N</code> ) under <code>--General--</code>                           |
| Fewer TSs in <code>FINAL_HL</code> than in <code>FINAL_LL</code> | Normal: some LL TSs don't converge to a real saddle point at HL, or exceed the <b>Energy</b> threshold from <code>--Kinetics--</code> | Not an error — check <code>tsdirHL_&lt;molecule&gt;/TSs/</code> for the detail on each one                                     |
| The calculation takes much longer than LL                        | Expected: DFT with ORCA is orders of magnitude more expensive than PM7                                                                | Tune <code>runningtasks</code> to the available cores; be patient — in this example (13 TSs, 5-atom molecule) it took ~2 hours |
| An ORCA job hangs indefinitely                                   | The low <code>timeout</code> from the LL page (e.g. 1000 s) is still set, too short for DFT                                           | Remove it or raise it substantially before launching HL (see above)                                                            |

### 3.9 What's next

With a complete reaction network refined at DFT level, and its corresponding kinetics, the next page covers how to refine the same TSs with an MLIP backend (UMA) instead of ORCA, comparing cost vs. accuracy. This same workflow (the LL and HL pages) can also be applied to a molecule other than FA.

## Chapter 4

# High-level (HL) calculations with UMA (MLIP)

Continuation of the ORCA high-level page: this page refines the same LL reaction network, but using a **machine-learning interatomic potential (MLIP)** — UMA — instead of a traditional DFT calculation.

### 4.1 What changes with MLIP vs. ORCA

The goal is the same as on the previous page: refine the TSs already discovered at LL. The difference is the engine doing the refinement — instead of a DFT calculation with ORCA, this uses **UMA**, a neural-network model trained to predict energies and forces directly, without solving any quantum-mechanical equation.

### 4.2 Requirements

- The LL page completed (`tsdirLL_FA/tslist` already generated).
- UMA model downloaded to `$AMK/models/` (see the installation page, step 7.3): `uma-m-1p1.pt` + `uma-m-1p1_atom_refs.yaml`.
- Python libraries installed (same page, step 7.3): `torch`, `sella`, `omegaconf`, `fairchem-core`.

### 4.3 Adjusting the `amk.dat`

Reuse the same `amk.dat` from the LL and ORCA pages, changing only the `HighLevel` keyword:

```
HighLevel mlip uma
```

Nothing else needs to change — `HL_rxn_network`, `IRCpoints`, `charge`, `mult`, and everything under `--Screening--/--Method--` stay the same.

**Note:** One real MLIP-specific constraint: `--Kinetics--` must use `Energy <value>`, not `Temperature`, because MLIP models don't compute the Gibbs free-energy correction needed to sort by Boltzmann weighting at a given temperature — with `Energy`, AutoMeKin sorts using E+ZPE instead. The example `amk.dat` already uses `Energy 150`, so no change is needed there.

## 4.4 Launching the calculation

The exact same command as with ORCA:

```
nohup hlcalcs.sh FA.dat 10 > hlcalcs.log 2>&1 &
```

But **what the 10 (runningtasks) means is different**. With ORCA, it's 10 concurrent ORCA processes. With MLIP, if a GPU is available, **runningtasks** is ignored entirely and **one process per GPU** is used (as many as `torch.cuda.device_count()` detects); if there's no GPU, the work is split across **runningtasks** CPU processes instead, each loading its own copy of the model — this is automatically capped so it doesn't exceed the available RAM (UMA's checkpoint needs a sizeable amount of memory per worker while loading).

## 4.5 GPU vs. CPU

For the formic acid example: **with a GPU, UMA took ~7 minutes** to refine the 13 TSs from the LL page (vs. ~1h50min with ORCA). **Without a GPU, it took ~14.6 minutes** — slower than with a GPU, but still much faster than ORCA. A GPU should be prioritized for this kind of calculation when available; CPU is still a reasonable fallback otherwise.

## 4.6 Where results live

Same as LL/HL, but with a different folder name: since `HighLevel` is `mlip`, the final folder is called `FINAL_ML_<molecule>/`, not `FINAL_HL_<molecule>/`. The rest of the layout (`tsdirHL_<molecule>/TSs`, `IRC`, `MINs`, `PRODs`, `KMC`) is identical to the ORCA page's.

## 4.7 Full verified example: FA with UMA

**RXNet** — 12 TSs survived refinement (out of the 13 candidates from LL):

| TS # | DE(kcal/mol) | Reaction path information |                |      |          |
|------|--------------|---------------------------|----------------|------|----------|
| ==== | =====        | =====                     |                |      |          |
| 1    | 7.4          | MIN                       | 1 <--->        | MIN  | 2        |
| 5    | 62.6         | MIN                       | 1 ---->        | PR1: | CO + H2O |
| 8    | 69.0         | MIN                       | 4 ---->        | PR1: | CO + H2O |
| 11   | 103.6        | PR3:                      | CHO + HO <---> | PR3: | CHO + HO |
| 12   | 111.4        | PR1:                      | CO + H2O <---> | PR2: | CO2 + H2 |

**rxn\_kin.txt** — 67.6% of the flux ends at CO+H<sub>2</sub>O, 7.8% at H<sub>2</sub>+CO<sub>2</sub>, the rest trapped in the MIN 3 isomer:

```
1 3 0.245852
1 CO+H2O 0.675716
3 CO+H2O 0.0784314
```

The main barrier (MIN1→CO+H<sub>2</sub>O, 62.6 kcal/mol) lands very close to what ORCA gave on the previous page (65.7 kcal/mol) — a difference of only ~3 kcal/mol, at a fraction of the computational cost.

## 4.8 Common troubleshooting

| Symptom                                                                                                  | Cause                                                                                                                                                                                                      | Fix                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MLIP model file not found: ...                                                                           | <code>uma-m-1p1.pt</code> is missing from <code>\$AMK/models/</code> , or the name doesn't match exactly                                                                                                   | Check the installation page, step 7.3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <code>_pickle.UnpicklingError: Weights only load failed when loading UMA</code>                          | The installed <code>e3nn</code> version loads its own <code>constants.pt</code> file without <code>weights_only=False</code> , and PyTorch $\geq 2.6$ no longer allows that by default                     | <code>E3NN_WIGNER=\$(python3 -c "import e3nn, os; print(os.path.join(os.path.dirname(e3nn.__file__), 'o3', '_wigner.py'))"); sed -i "s/'constants.pt')/'constants.pt'), weights_only=False)/" "\$E3NN_WIGNER"</code> — then delete <code>tsdirHL_&lt;molecule&gt;/</code> and <code>FINAL_ML_&lt;molecule&gt;/</code> and rerun <code>export TORCHDYNAMO_DISABLE=1</code> before launching (add it to <code>~/.bashrc</code> to make it permanent); delete <code>tsdirHL_&lt;molecule&gt;/</code> and <code>FINAL_ML_&lt;molecule&gt;/</code> and rerun |
| <code>AMK_ERROR: CppCompileError / g++: error: unrecognized command line option '-std=c++20'</code>      | UMA tries to compile the model with <code>torch.compile</code> , and the installed <code>g++</code> is too old for the <code>-std=c++20</code> flag                                                        | <code>export TORCHDYNAMO_DISABLE=1</code> before launching (add it to <code>~/.bashrc</code> to make it permanent); delete <code>tsdirHL_&lt;molecule&gt;/</code> and <code>FINAL_ML_&lt;molecule&gt;/</code> and rerun                                                                                                                                                                                                                                                                                                                                 |
| Limiting CPU workers to N (requested M)... appears, and fewer workers run than <code>runningtasks</code> | Without a GPU, each worker loads its own full copy of the model into RAM (peaks at ~24 GB for UMA while loading, ~2 GB for MACE); the worker count is automatically reduced to avoid running out of memory | Not an error — this is the expected safeguard; with plenty of RAM to spare, free up memory or use a machine with more RAM to allow more workers                                                                                                                                                                                                                                                                                                                                                                                                         |
| The run is very slow without a GPU                                                                       | Expected: CPU is slower than GPU (see above)                                                                                                                                                               | Use a GPU if available; otherwise be patient — it's still faster than ORCA                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| IRC inner loop failed to converge! warning                                                               | One particular TS doesn't give a clean IRC at the MLIP level                                                                                                                                               | Not fatal — AutoMeKin discards that path and continues; check whether that TS still shows up in the final <code>RXNet</code>                                                                                                                                                                                                                                                                                                                                                                                                                            |
| The final folder is called <code>FINAL_ML_FA</code> , not <code>FINAL_HL_FA</code>                       | Different name for <code>HighLevel mlip</code>                                                                                                                                                             | Expected behavior — look for <code>FINAL_ML_&lt;molecule&gt;/</code>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

## 4.9 What's next

With ORCA and UMA, there are now two ways to refine the same LL network, with a clear cost/accuracy trade-off. The same workflow (just swapping `HighLevel mlip uma` for `HighLevel mlip mace`) works to try MACE, the other supported MLIP model. The next page covers how to visualize and analyze these results with `amk_tools`.

## Chapter 5

# Visualization and analysis with amk\_tools

Continuation of the previous two pages: this page installs `amk_tools`, the companion tool for **visualizing** and **analyzing** the reaction networks AutoMeKin generates, and walks through all of its features on the real results from the ORCA/DFT and UMA pages.

### 5.1 What `amk_tools` is, and why it lives in a separate environment

`amk_tools`, by Diego Garay, doesn't run any calculations — it's purely a reader/visualizer. It reads the plain-text files and SQLite databases already present inside a `FINAL_LL_*`, `FINAL_HL_*`, or `FINAL_ML_*` folder, and from that builds:

- an **interactive visualization** of the reaction network (HTML, navigable in any browser), and
- **statistics** on that network's topology (number of nodes/edges, average path length, clustering coefficient, etc., compared against an equivalent random graph).

**Note: Why it needs its own conda environment, separate from `amk_env`:** `amk_tools` pins `bokeh<3.0.0`, and due to that Bokeh version's own dependencies, requires `numpy<2`. If the MLIP models (MACE/UMA) were installed on the installation page (step 7.3), `amk_env` has `numpy>=2` — a combination `amk_tools` simply won't start with (`AttributeError: module 'numpy' has no attribute 'bool8'`). Rather than forcing a version combination that breaks one thing or the other, the cleanest approach is a completely separate `amk_view` environment, just for this. This is safe because `amk_tools` **never interacts with AutoMeKin itself** — it only reads already-generated files. Installing, upgrading, or even deleting `amk_view` entirely has zero effect on the AutoMeKin installation or on `amk_env`.

### 5.2 Isolated installation

```
conda create -n amk_view python=3.11 -y
conda activate amk_view
pip install git+https://github.com/dgarayr/jsmol_to_bokeh.git
pip install --no-deps git+https://github.com/dgarayr/amk_tools.git
pip install "bokeh>=2.3.2,<3.0.0" "numpy<2" networkx matplotlib scipy
```

`pip install` only provides the importable `RXReader/RXVisualizer` modules — it doesn't install the command-line tools (`amk_gen_view.py`, `amk_rxn_stats.py`), since the package doesn't declare them as entry points. Fetch those separately into this same environment's `bin` folder:

```
curl -L -o $CONDA_PREFIX/bin/amk_gen_view.py
↪ https://github.com/dgarayr/amk_tools/raw/master/scripts/amk_gen_view.py
curl -L -o $CONDA_PREFIX/bin/amk_rxn_stats.py
↪ https://github.com/dgarayr/amk_tools/raw/master/scripts/amk_rxn_stats.py
chmod +x $CONDA_PREFIX/bin/amk_gen_view.py $CONDA_PREFIX/bin/amk_rxn_stats.py
```

Verify both commands respond:

```
amk_gen_view.py --help
amk_rxn_stats.py --help
```

From here on, generating a visualization or statistics only requires `conda activate amk_view` — `amk_env` or AutoMeKin do **not** need to be active at the same time.

## 5.3 The two tools: overview

| Tool                          | What it does                                                                                         | Output                          |
|-------------------------------|------------------------------------------------------------------------------------------------------|---------------------------------|
| <code>amk_gen_view.py</code>  | Builds an interactive graph of the reaction network (nodes = minima/products, edges = TSs)           | A self-contained .html file     |
| <code>amk_rxn_stats.py</code> | Computes topological properties of that same network and compares them to an equivalent random graph | A .txt file with the statistics |

Both are invoked the same way: first the `FINAL_*` folder with the results, then which version of the network to use.

## 5.4 amk\_gen\_view.py: complete option reference

```
amk_gen_view.py <finaldir> <rxnfile> [options]
```

**Positional (required):**

| Argument              | Meaning                                                                                                                                                           |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>finaldir</code> | The results folder: <code>FINAL_LL_&lt;molecule&gt;</code> , <code>FINAL_HL_&lt;molecule&gt;</code> , or <code>FINAL_ML_&lt;molecule&gt;</code>                   |
| <code>rxnfile</code>  | Which version of the network to use: <code>RXNet</code> (full), <code>RXNet.cg</code> (connected graph), or <code>RXNet.rel</code> (already filtered/relativized) |

**“RXN parsing” group — what to include and how to reference energies:**

| Option                             | Short            | What it does                                                                                                     |
|------------------------------------|------------------|------------------------------------------------------------------------------------------------------------------|
| <code>--barrierless</code>         | <code>-b</code>  | Adds the barrierless channels from <code>RXNet.barless</code> to the graph (otherwise they’re left out entirely) |
| <code>--vibrations NVIBR</code>    | <code>-v</code>  | Number of normal vibrational modes to include in the visualization; <code>-1</code> for all                      |
| <code>--ref_state REF_STATE</code> | <code>-rs</code> | Switches the reference state (energy zero) to a different node than the one AutoMeKin used by default            |

**“Path handling” group — building paths/energy profiles:**

| Option                                     | Short            | What it does                                                                                                                                                                                                                                                                                                                                                      |
|--------------------------------------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>--paths [SOURCE [TARGET ...]]</code> | <code>-p</code>  | Generates paths. <b>With no arguments</b> , builds every possible profile starting from the <b>global minimum</b> (AutoMeKin's internal reference node) — this is exactly what's used in the example below. With a <b>SOURCE</b> , filters paths connected to that node. With both <b>SOURCE</b> and <b>TARGET</b> , searches specifically for paths between them |
| <code>--cutoff_path CUTOFF</code>          | <code>-c</code>  | Maximum search depth when <b>SOURCE+TARGET</b> are used together (default 4)                                                                                                                                                                                                                                                                                      |
| <code>--efilter EFILTER</code>             | <code>-e</code>  | Energy threshold (kcal/mol) for discarding paths; no filter by default                                                                                                                                                                                                                                                                                            |
| <code>--unreduced</code>                   | <code>-u</code>  | Generates the full graph, without excluding nodes that don't appear in any path                                                                                                                                                                                                                                                                                   |
| <code>--geomolden</code>                   | <code>-ng</code> | Updates the displayed geometries from MOLDEN files                                                                                                                                                                                                                                                                                                                |

### “File handling” group — output name and title:

| Option                         | Short           | What it does                                                   |
|--------------------------------|-----------------|----------------------------------------------------------------|
| <code>--outfile OUTFILE</code> | <code>-o</code> | Output HTML filename (default <b>network.html</b> )            |
| <code>--title TITLE</code>     | <code>-t</code> | Visualization title (default “Reaction network visualization”) |

### “Aspect handling” group — appearance:

| Option                               | Short           | What it does                                                                                                                                      |
|--------------------------------------|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>--resolution RESOLUTION</code> | <code>-r</code> | HTML size, <b>WIDTH,HEIGHT</b> in pixels (default 1400,800)                                                                                       |
| <code>--fasterlayout</code>          | <code>-f</code> | Uses the faster layout algorithm ( <b>nx.spring_layout</b> ) instead of the default ( <b>nx.kamada_kawai_layout</b> ) — useful for large networks |

## 5.5 Practical example: network from the global MIN, DFT vs UMA

This builds the network from the global minimum (`--paths`, no arguments — see the table above), using `RXNet.rel` and including the barrierless channels, for the two HL results from the previous two pages.

### With the ORCA/DFT results:

```
cd test_example_dft
amk_gen_view.py FINAL_HL_FA RXNet.rel --paths --barrierless --outfile
↪ network_rel_with_barrierless.html
```

### With the UMA results:

```
cd test_example_uma_cpu
amk_gen_view.py FINAL_ML_FA RXNet.rel --paths --barrierless --outfile
↪ network_rel_with_barrierless.html
```

Both commands are identical except for the input folder — that's how simple it is to visualize the

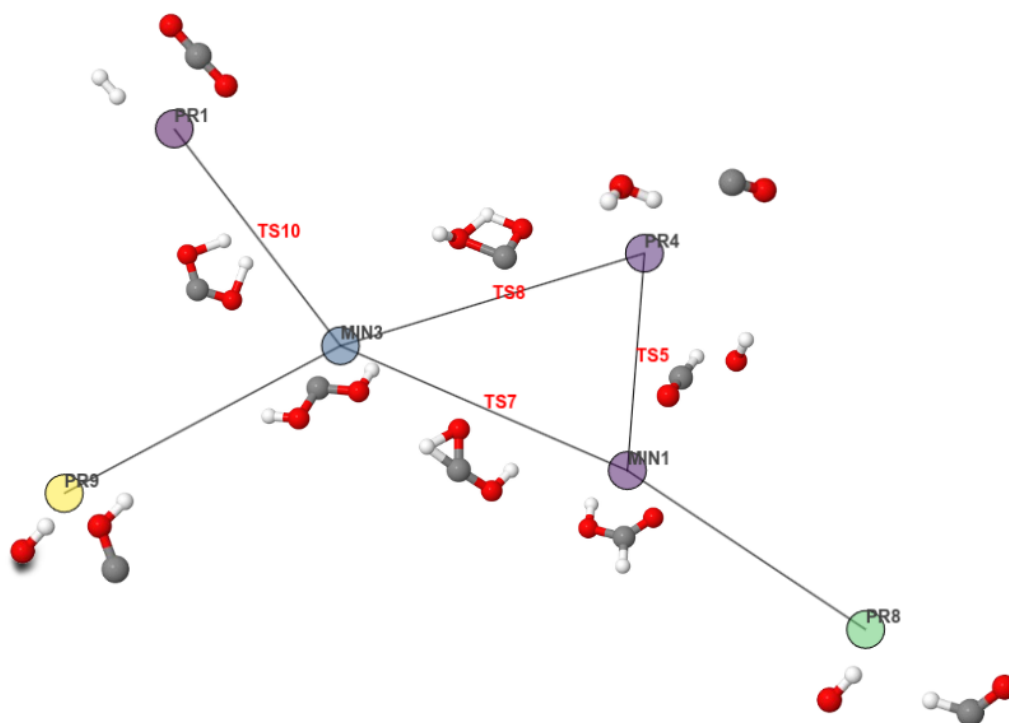


Figure 5.1: Reaction network generated from RXNet.rel with UMA-refined results

same system computed with two different HL methods (each lives in its own folder, so using the same filename for both doesn't clash).

This is the network generated from `RXNet.rel` with the results refined using UMA: each node is a species (minima or products) and each edge is a transition state connecting them, positioned relative to the global minimum's energy.

The generated HTML file also includes an energy profile of the reaction network, viewable by clicking the “Show profile” tab. In the image above, **PR8** and **PR9** are the barrierless products, obtained and stored in `RXNet.barrless`.

**Note:** Product numbers (PR#) may not match between different runs. That's expected, not an error: LL sampling launches its MD trajectories from randomized starting conditions, so which minima and products get discovered — and in what order — can differ slightly between runs, even for the same molecule and input file. The PR numbering is just the order of discovery, so it shifts accordingly; the actual topology and energies are what matter, and those are reproducible.

Opening both HTML files in a browser and comparing them shows the same topology (what connects to what), but the barrier heights differ as seen on the previous two pages.

## 5.6 `amk_rxn_stats.py`: network statistics

```
amk_rxn_stats.py <finaldir> [options]
```

| Option                          | Short            | What it does                                                                               |
|---------------------------------|------------------|--------------------------------------------------------------------------------------------|
| <code>--rxnfile RXNFILE</code>  | <code>-r</code>  | Which network version to use (default <code>RXNet.cg</code> )                              |
| <code>--skip_barrierless</code> | <code>-sb</code> | Excludes the <code>RXNet.barrless</code> channels (included by default if the file exists) |

| Option                                 | Short            | What it does                                                                             |
|----------------------------------------|------------------|------------------------------------------------------------------------------------------|
| <code>--Ngraphs NGRAPHS</code>         | <code>-Ng</code> | Number of equivalent random Erdős-Rényi graphs to generate for comparison (default 1000) |
| <code>--outfilename OUTFILENAME</code> | <code>-o</code>  | Output filename (default <code>rxn_stats.txt</code> )                                    |

Computes: number of nodes and edges, average shortest path length, clustering coefficient, transitivity, edge density, and degree assortativity coefficient — all compared against the average of **Ngraphs** random graphs of the same size, to show whether a reaction network is “more ordered” or “more random” than would be expected by pure chance.

**Real example** (on the ORCA/DFT results):

```
cd test_example_dft
amk_rxn_stats.py FINAL_HL_FA --rxnfile RXNet.rel --outfilename FA_DFT_stats.txt
```

```
Number of nodes = 6
Number of edges = 7
Average shortest path length of the current network = 1.6
Average shortest path length of the equivalent random network = 2.11
Average clustering coefficient of the current network = 0.44
Average clustering coefficient of the equivalent random network = 0.36
Transitivity of the current network = 0.43
Transitivity of the equivalent random network = 0.37
Density of edges (edges/possible_edges) = 0.47
Degree assortativity coefficient = -0.70
```

FA’s network has a shorter average path (1.6 vs. 2.11) and more edge density than an equivalent random graph — expected for a real reaction network of a small molecule, where minima tend to be well interconnected rather than randomly scattered.

## 5.7 Common troubleshooting

| Symptom                                                                                                        | Cause                                                                                                                                                                                         | Fix                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ModuleNotFoundError: No module named 'RXVisualizer'</code>                                               | <code>pip install</code> only provides the modules, not the command-line scripts                                                                                                              | Download <code>amk_gen_view.py/amk_rxn_stats.py</code> with <code>curl</code> as shown above                                                                                                     |
| <code>AttributeError: module 'numpy' has no attribute 'bool8'</code><br>The HTML comes out blank or very small | <code>amk_env</code> is active (with <code>numpy&gt;=2</code> from the MLIP setup) instead of <code>amk_view</code><br>Network with very few nodes, or <code>--efilter</code> too restrictive | <code>conda activate amk_view</code> before running any <code>amk_tools</code> command<br>Try without <code>--efilter</code> , or check that <code>finaldir/rxnfile</code> point to real results |
| <code>finaldir</code> must be passed from <code>amk_rxn_stats.py</code>                                        | Missing the required <code>finaldir</code> positional argument                                                                                                                                | <code>amk_rxn_stats.py &lt;FINAL_folder&gt;</code> — it’s required, unlike the options                                                                                                           |

## 5.8 What’s next

With this, the full cycle is complete: install AutoMeKin, explore at low level, refine at high level with DFT or an MLIP, and visualize and analyze the result. From here, the natural next step is applying the whole workflow to a molecule other than FA.

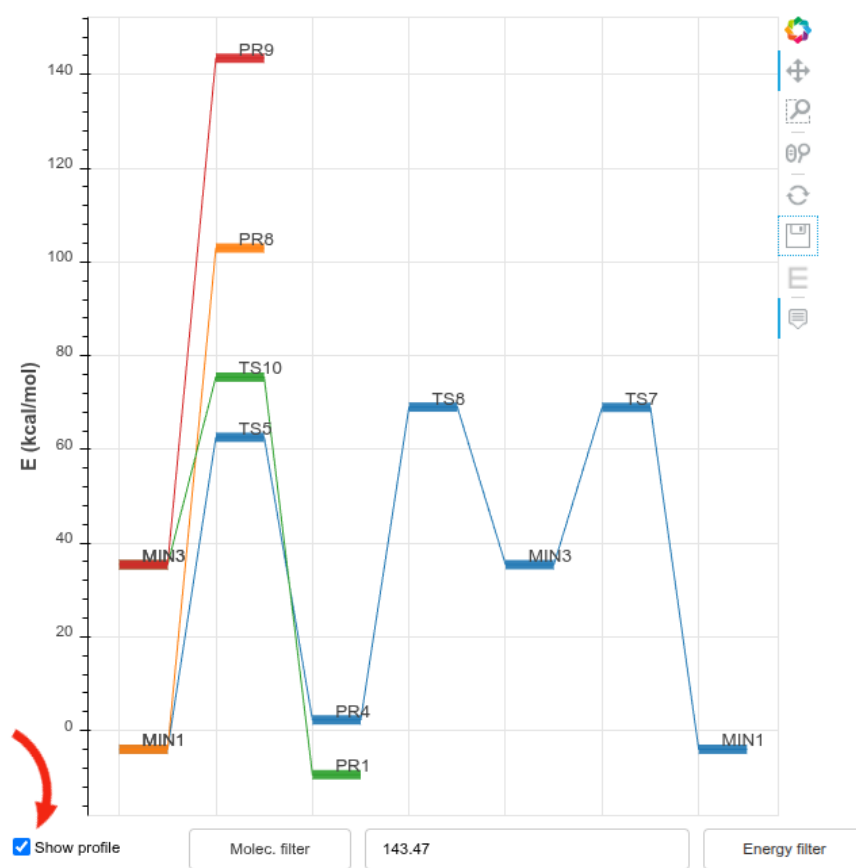


Figure 5.2: Energy profile of the reaction network